Non-Graded Project: TechCorp Enterprise LLM Launch Playbook

Launching TechCorp's Confidence Engine: From architecture choice to production reliability

Learning Objectives

By the end of this project, learners will be able to:

- Select an LLM architecture that balances accuracy, latency, and cost for TechCorp's dual-intent assistant.
- Configure LoRA-based fine-tuning workflows that raise policy accuracy while controlling GPU spend.
- Design a secure, observable deployment pipeline with autoscaling, monitoring, and incident readiness.
- Communicate operational trade-offs and mitigation plans to business and compliance stakeholders.

Description

In this course-end project, you will design, adapt, and operationalize a modular LLM platform for TechCorp that spans architecture selection, parameter-efficient fine-tuning, and production deployment safeguards. You will synthesize decisions made across the course to deliver a launch-ready plan backed by measurable evidence.

Structure of the Activity

Dataset Information/Resources Required

This project synthesizes artifacts from Module 1 (architecture evaluation), Module 2 (fine-tuning playbook), and Module 3 (deployment topology). You will reference your completed lab work and course materials to create a comprehensive enterprise launch plan.

Tools Used:

- Hugging Face Transformers
- PEFT/LoRA
- Docker, Kubernetes
- Linux Desktop Image
- Jupyter Notebook
- Prometheus/Grafana
- Locust/k6
- Git

Setup Requirements

- Completed labs from Modules 1, 2, and 3
- Access to course materials and readings
- Presentation software (PowerPoint, Google Slides, or equivalent)
- Document editing software (Word, Google Docs, or Markdown editor)
- Diagramming tool (draw.io, Lucidchart, or equivalent)

Scenario

TechCorp's executive team approved a go-live date for its customer-facing LLM assistant. You must finalize the end-to-end plan: confirm the architecture baseline, document fine-tuning evidence, secure the deployment, and define observability plus rollback strategies. Funding is contingent on proving sub-200 ms latency, $5,000 monthly inference costs, and 95% policy adherence. Compliance has requested auditable controls before sign-off.

Objective

Deliver a launch-ready plan backed by measurable evidence that synthesizes architecture selection, fine-tuning workflows, and deployment safeguards into a comprehensive enterprise LLM platform.

coursera

Instructions for Participants

Step 1: Summarize Architecture Decision

Objective

Present the evaluation matrix comparing encoder-only, decoder-only, and encoder-decoder options.

Task:

Create a comprehensive architecture evaluation matrix that highlights latency, accuracy, token cost, and mitigation levers (quantization, caching). Reference your Module 1 work and expand it with executive-level justifications.

Expected Outcome: A justified architecture selection with SLA alignment.

Tips:

- Include quantitative metrics: p95 latency, accuracy percentages, and monthly cost projections
- Document assumptions (hardware, batch size, traffic patterns)
- Explain why the selected architecture meets TechCorp's <200ms latency and $5,000/month budget requirements
- Show trade-offs between architectures with clear reasoning

Step 2: Document Fine-tuning Strategy

Objective

Outline the LoRA configuration, dataset preparation, validation metrics, and cost savings.

Task

Develop a reusable fine-tuning playbook that documents your LoRA adapter configuration, training approach, policy accuracy improvements, and compliance safeguards. Build upon your Module 2 lab work.

Expected Outcome: A reusable fine-tuning playbook with compliance checkpoints.

Tips

- Document LoRA hyperparameters (rank, alpha, target modules, dropout)
- Show before/after metrics: policy accuracy improvement (must meet 95% target)
- Include data preparation steps with privacy safeguards
- Calculate GPU cost savings from parameter-efficient tuning
- Add validation checkpoints and compliance review gates

Step 3: Design Deployment Topology

Objective

Sketch the Kubernetes-based microservice architecture, ingress controls, autoscaling policies, and monitoring stack.

Task

Create a deployment topology diagram of your Kubernetes infrastructure, including security layers, autoscaling configuration, and observability components. Build upon your Module 3 lab work.

Expected Outcome: Diagram plus configuration notes showing layered defenses and observability.

Tips

- Show all security layers: container hardening, RBAC, ingress with mTLS, prompt filtering
- Document autoscaling triggers (CPU, memory, custom metrics)
- Include monitoring stack components (Prometheus, Grafana, alert rules)
- Label key components: pods, services, ingress, persistent volumes
- Add alt text to diagrams for accessibility

coursera

Step 4: Create Launch Readiness Runbook

Objective

Author incident response steps, alert thresholds, rollback paths, and stakeholder communications.

Task

Write a concise launch-readiness runbook that operations teams can use during go-live and for ongoing production support. Include both technical procedures and communication protocols.

Expected Outcome: A concise runbook referencing metrics and guardrails.

Tips

- Define alert thresholds: latency (p95 > 200ms), error rate (>1%), pod crashes
- Document rollback procedures with step-by-step commands
- List escalation paths and notification channels (PagerDuty, Slack, email)
- Include troubleshooting decision trees for common incidents
- Reference monitoring dashboards and log locations

Step 5: Record Executive Summary

Objective

Produce a 2-minute narrative summarizing the plan, trade-offs, and next steps.

Task

Write an executive summary that communicates your technical decisions in business terms. This narrative should be suitable for leadership review and stakeholder buy-in.

Expected Outcome: Stakeholder-ready message linking decisions to business value.

Tips

- Connect technical choices to business outcomes (cost savings, customer satisfaction, compliance)
- Highlight how the solution meets the three critical requirements: <200ms latency, $5,000/month budget, 95% policy adherence
- Address key risks and mitigation strategies
- Outline next steps and timeline for production rollout
- Keep language accessible for non-technical stakeholders

Compile Your Findings

1. Deliverable 1: Architecture Evaluation Matrix

Deliverable: Upload your architecture evaluation matrix (PDF or slide).

Include:

- Comparison of encoder-only, decoder-only, and encoder-decoder architectures
- Latency metrics (p95) for each architecture
- Accuracy scores across key intents

- Monthly cost projections
- Mitigation strategies (quantization, caching, retrieval)
- Clear justification for selected architecture
- Documented assumptions and constraints

File Types: .pdf, .pptx, .ppt, .key

2. Deliverable 2: LoRA Fine-tuning Playbook

Deliverable: Provide your LoRA fine-tuning playbook with metrics and compliance safeguards.

Include:

Include:

- LoRA configuration parameters (rank, alpha, target modules, dropout)
  Dataset preparation methodology
- Policy accuracy metrics (baseline vs. fine-tuned)
- GPU cost analysis and parameter reduction statistics
- Compliance checkpoints and validation gates
- Training procedure documentation
- Privacy safeguards for sensitive data

File Types: .pdf, .docx, .md

3. Deliverable 3: Deployment Topology Diagram

Deliverable: Submit your deployment topology diagram and observability checklist.

Include:

- Kubernetes architecture diagram showing all components
- Security layers (container hardening, RBAC, ingress controls, mTLS)
- Autoscaling configuration (HPA triggers, resource limits)
- Monitoring stack (Prometheus, Grafana, alert definitions)
- Prompt filtering and response sanitization middleware

- Observability checklist with key metrics
- Alt text for all diagram elements (accessibility requirement)

File Types: .pdf, .png, .svg, .pptx

4. Deliverable 4: Launch Readiness Runbook and Executive Summary

Deliverable: Paste your launch readiness runbook and executive summary narrative.

Include:

Include:

Launch Readiness Runbook:

- Alert thresholds and definitions
- Incident response procedures
- Rollback commands and procedures
- Escalation paths and notification channels
- Troubleshooting decision trees

Executive Summary:

- Overview of architecture selection and rationale
- Fine-tuning approach and policy accuracy gains
- Deployment security and observability highlights
- How solution meets requirements (<200ms latency, $5,000/month, 95% policy adherence)
- Key risks and mitigation strategies
- Business value and customer impact
- Next steps and rollout timeline

Response Type: Rich Text (paste into submission form)

Best Practices for Submission

Be Specific About Metrics:

- Include quantitative data: latency measurements, accuracy percentages, cost projections
- Cite tooling versions (e.g., Transformers 4.35, PyTorch 2.1)
- Connect mitigations to quantified outcomes

Document Assumptions:

- Hardware specifications (GPU type, memory, CPU)
- Traffic patterns and user load estimates
- Data characteristics (token limits, conversation length)

Ensure Accessibility:

- Add alt text to all diagrams and visuals
- Use sufficient color contrast (meet WCAG guidelines)
- Provide text descriptions of visual information

Label Clearly:

- Each artifact should clearly reference TechCorp's constraints
- Balance technical depth with executive clarity
- Use consistent terminology across all deliverables

File Format Guidelines:

- Use vector-friendly formats for diagrams (.svg, .pdf)
- Provide accessible PDFs for documents
- Include alternate versions where helpful (e.g., .docx + .pdf)

Quality Assurance Checklist

Before submitting, verify that:

- Architecture matrix references latency, accuracy, context, and cost evidence
- LoRA playbook includes dataset prep, adapter settings, validation metrics, and compliance notes
- Deployment diagram lists security layers, autoscaling triggers, and monitoring metrics
- Runbook documents alerts, rollback procedures, and communication pathways
- Executive summary ties decisions to TechCorp's business outcomes

- All metrics align with current pricing and performance benchmarks
- Diagrams include alt text for accessibility
- Files are clearly labeled and properly formatted

Summary

This course-end project synthesizes your learning across LLM architecture selection, parameter-efficient fine-tuning, and production deployment. By completing this comprehensive launch playbook, you demonstrate the ability to make data-driven technical decisions, balance competing constraints, and communicate effectively with both technical and business stakeholders. The deliverables you create represent real-world artifacts used in enterprise LLM deployments.

Key competencies demonstrated:

- Systematic evaluation of LLM architectures against enterprise requirements
- Implementation of cost-effective fine-tuning strategies with compliance safeguards
- Design of secure, observable production infrastructure
- Clear communication of technical decisions to diverse stakeholders

Resources

Primary Materials:

- Module 1 Lab: Select and Evaluate LLM Architectures for TechCorp
- Module 2 Lab: Fine-tune a Customer Service Model for RetailCorp
- Module 3 Lab: Deploy and Monitor an LLM Application for HealthTech Inc
- Course readings and video lectures from all three modules

Reference Documentation:

- Hugging Face Transformers and PEFT documentation
- Kubernetes deployment best practices
- Prometheus and Grafana monitoring guides
- LLM architecture comparison research papers

Additional Resources:

- Enterprise SLA and compliance frameworks
- LoRA in Production: Microsoft's approach to efficient model adaptation
- Anthropic's Production Security: Lessons from Claude's Deployment
- WCAG accessibility guidelines for technical documentation

Submission Platform:

- Course LMS submission portal with AI-graded assessment

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.